Toronto Fresh Food (TFF)

Local LLM AI Agent - README

June 1, 2026

1. Choosing the Best Local LLM Model to Download

Before choosing a local LLM runner, we have to first choose a model that we have to download. The following section goes over which local LLM model is best for the company's needs.

To choose the best local LLM model for the company, we need to look at the company's needs and restrictions. We ideally want a local LLM model that is best suited for agentic tasks and is able to create apps with great user interfaces (UI) and user experiences (UX) through powerful UI/UX generation. We are restricted to 32 GB of video random access memory (VRAM), meaning that powerful local LLM models like Kimi-K2.6 will not be possible to run on our devices because they require 250+ GB of VRAM. Based on the needs above, the best local LLM model to choose would be **Qwen3.5-27B** or Qwen3-Coder 30B (A3B) since they excel at handling multi-file refractors, understanding complex project structures and producing high quality UI code while also being good at agentic tasks. It would make the most sense to run these local LLM models on Q4_K_XL or 5/6 quants because they would be able to best compress the model so that it can run smoothly on just 32 GB of VRAM.

2. Choosing the Best Local LLM Runner

Now that we have chosen the best Local LLM model to use, we now need to choose the best local LLM runner for these models. I recommend **LM Studio** because it is the most user-friendly interface, allows for you to search for and download quantized GPT-generated unified formatted (GGUF) models directly, and tells you whether or not a local LLM fits within the memory on your device. The following YouTube video showcases how to run a local LLM model on LM Studio.

How do we run Qwen3.5-27B in LM Studio?

1. Download the model
    1. Open LM Studio and use the search bar to search for:

```
unsloth Qwen3.5-27B-GGUF
```

    2. Select the repository (usually by Unsloth) and locate the Q4_K_XL or UD-Q4_X_L file variant
    3. Download the GGUF File

2. Configure system settings

    1. GPU offloading: open the settings panel on the right, enable GPU offload, and set it to the max (99-100) to maximize the GPUs VRAM

    2. Context Length: set the desired context length. The model supports up to 256K, but you should adjust this based on your available memory

    3. KV Cache: if memory issues arise, consider using q8_0 for your K and V cache, or reduce the context length

3. Run the model: load the model in the chat tab

3. Orchestration (Tools Connection)

If we were to use the following setup of running Qwen3.5-27B on Q4_K_XL quants on LM Studio, we could orchestrate the local LLM using the following process:

1. Enable the LM Studio Server

    - Navigate to the Developer/Local Server tab (represented by the <-> icon)
    - Click “Start Server”
    - Note your base URL. The default URL is usually http://localhost:1234/v1

2. Choose Your Orchestration Framework

    - Since our local server mimics OpenAI, we can use industry-standard orchestration tools. Since we want Retrieval-Augmented Generation (RAG) pipelines, using **LangChain** would be the most efficient option

3. Connect the Orchestrator via Code

    - I recommend using **Python** due to its simplicity and its compatibility because it is the most commonly used programming language in data science and data engineering
    - You need to install Python SDK and LangChain with its Partner Packages using your command prompt by typing the following commands (note: you will need to have Python installed on your machine for this to work):

```
pip install lmstudio

pip install langchain

pip install langchain-openai

pip install langchain-community

pip install langchain-chroma
```

The vanilla Python script you should run to connect LangChain to LM Studio is:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

# Point LangChain to LM Studio
llm = ChatOpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio",
    model="qwen3.5-27b",
    temperature=0.7
)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful orchestration
assistant."),
    ("user", "{input}")
])

chain = prompt | llm
print(chain.invoke({"input": "Explain how you will
manage my tasks."}))
```

- Because LM Studio's local servers mimics OpenAI's API, you can just simply use LangChain's OpenAI classes and point them to your localhost
- This code is used to build tool-calling chains or RAG

4. Vector Database (Storage)

Using the same system above, here is how you can build a vector database.

1. Download an embedding model in LM Studio

    - Open LM Studio and go to the search tab
    - Choose an highly-rated embedding model, I recommend **nomic-embed-text-v1.5**
    - Download a GGUF file for the embedding model
    - Go to the “Local Server" tab in LM Studio. Ensure that you load both the Qwen model and your embedding model (LM studio allows for multiple models to be loaded within the server settings)

2. Set Up Your LangChain Environment

    - You will need to install a vector store library on your device. **ChromaDB** is one of the easiest ones to use with Python. To download it, open your command prompt and install it using the following command:

      ```
      pip install chromadb langchain-openai
      langchain-community sentence-transformers
      ```

    - You will also need to install HuggingFaces’ sentence transformers in the same manner:

      ```
      pip install langchain-huggingface
      sentence-transformers
      ```

3. Write the Python Code to Build the Database

    The vanilla Python script you should run to build the vector database is:

    - We will use metadata filtering to ensure that all 8 of the departments: marketing, business development, production, accounting, HR, QA, engineering and warehousing will only receive answers that are retrieved from their departments documents in separate RAG pipelines

```
from langchain_huggingface import
HuggingFaceEmbeddings

# This downloads a small, fast, and popular
open-source embedding model
embeddings =
HuggingFaceEmbeddings(model_name="all-MiniLM
-L6-v2")

from langchain_openai import ChatOpenAI
from langchain_community.vectorstores import
Chroma

# 1. Initialize your LLM and Vector DB
LM_STUDIO_URL = "http://localhost:1234/v1"
llm = ChatOpenAI(
    openai_api_base=LM_STUDIO_URL,
    openai_api_key="lm-studio",
    temperature=0.3
)

# Assuming 'embeddings' is already defined
in your code
vector_db =
Chroma(persist_directory="./my_local_vectord
b", embedding_function=embeddings)

# 2. Define your list of departments exactly
as they appear in your metadata
departments = [
    "marketing",
    "business development",
    "production",
    "accounting",
    "HR",
    "QA",
    "engineering",
    "warehousing"
]
```

```
# 3. Create a dictionary to store a
dedicated retriever for each department
department_retrievers = {}

for dept in departments:
    dept_filter = {"department": dept}
    department_retrievers[dept] =
vector_db.as_retriever(search_kwargs={"k":
3, "filter": dept_filter})

# 4. Example Usage:
# If you want to search only within the HR
department's documents:
# hr_retriever = department_retrievers["HR"]
# hr_docs = hr_retriever.invoke("What is the
standard onboarding process?")
# If you want to search only within
Engineering:
# engineering_retriever =
department_retrievers["engineering"]
# eng_docs =
engineering_retriever.invoke("Where are the
deployment logs stored?")
```

5. Embedded Model (Data to a Vector)

Using the same system as before, here is how we can take the data and turn it into a vector.

1. Install the following package the same way you installed ChromaDB when making the vector database:

   ```
   pip install langchain langchain-openai langchain-community
   chromadb tiktoken
   ```

   - In order for all types of documents to work, we need to install a few more libraries using command prompt again:

     ```
     pip install unstructured pdf2image pdfminer.six
     pip install pi-heif
     pip install unstructured-inference
     ```

```
pip install docx2txt
```

2. Since we want to automate the document loading process, it makes the most sense to download Google Drive for desktop and point to our shared Google Drive folders. The following process explains how to achieve this:

    1. Download **Google Drive for desktop** onto your device

    2. Sign in using your TFF Google account

    3. Find the path: open Windows File Explorer and you will see a new drive (Google Drive (G :)). Navigate through it until you find the main company folder where you store your documents. Copy and save that path.

3. Write the following Python script to create a file called config.json:

```
import json
data = {
    "approved_departments": [
        "engineering",
        "HR",
        "marketing",
        "sales",
        "QA",
        "accounting",
        "business Development",
        "production",
        "warehousing"
    ]
}
#2. Open a file in write mode ('w') and save the
data
with open("config.json", "w", encoding = "utf-8")
as file:
    json.dump(data, file, indent = 4)
```

    - This new config.json file will look like this:

```
{
```

```
    "approved_departments": [
       "engineering",
       "HR",
       "marketing",
       "sales",
       "QA",
       "accounting",
       "business Development",
       "production",
       "warehousing"
    ]
}
```

- Now inside the main company folder, make 9 separate folders for all of these departments and store your data inside of them. Make sure that you use properly case sensitivities and abbreviations because if you don't then metadata filtering won't work correctly
- Open this file with a Notepad when you want to add another department to the list. Add the new department in the following manner where "newdepartment" is the name of the department that you want to add:

```
{
    "approved_departments": [
       "engineering",
       "HR",
       "marketing",
       "sales",
       "QA",
       "accounting",
       "business Development",
       "production",
       "warehousing"
       "newdepartment"
    ]
}
```

- Save the file in Notepad when you are done.
- Make a new folder called "newdepartment" and save your data in there

4. Write and run the following Python script to ingest your data into ChromaDB:

    - Note: G:/Shared drives/TFF_Docs should be replaced with the actual directory of the main company folder that you found earlier

```
import os
import json
from langchain_text_splitters import
RecursiveCharacterTextSplitter
from langchain_openai import
OpenAIEmbeddings
from langchain_chroma import Chroma

# --- Multi-Format Document Loaders ---
from langchain_community.document_loaders
import PyPDFLoader, TextLoader,
Docx2txtLoader

# --- 1. Load Approved Departments from JSON
Config ---
with open("config.json", "r",
encoding="utf-8") as file:
    config = json.load(file)
departments = config["approved_departments"]

# --- 2. Base Configuration ---
# Update this path if your shared Google
Drive has a different folder name!
BASE_DRIVE_PATH = "G:/Shared
drives/TFF_Docs"
PERSIST_DIRECTORY = "./local_chroma_db"

# Initialize your local embedding model via
LM Studio
embeddings = OpenAIEmbeddings(

openai_api_base="http://localhost:1234/v1",
    openai_api_key="lm-studio",
    model="nomic-embed-text-v1.5",
    check_embedding_ctx_length=False
)
```

```
all_processed_chunks = []

print("Starting automated Google Drive
document ingestion...")

# --- 3. Loop Through Each Department Folder
---
for dept in departments:
    # Build the path to the specific
department folder (e.g., G:/Shared
drives/TFF_Docs/HR)
    dept_folder_path =
os.path.join(BASE_DRIVE_PATH, dept)

    if os.path.exists(dept_folder_path):
        print(f"\nScanning files for
department: '{dept}'...")

        dept_documents = []

        # Scan the folder for all files
        for filename in
os.listdir(dept_folder_path):
            file_path =
os.path.join(dept_folder_path, filename)

            if os.path.isfile(file_path):
                try:
                    # Route the file to the
correct LangChain loader
                    if
filename.lower().endswith(".pdf"):

dept_documents.extend(PyPDFLoader(file_path)
.load())

                    elif
filename.lower().endswith(".txt"):
```

```
dept_documents.extend(TextLoader(file_path,
encoding="utf-8").load())

                    elif
filename.lower().endswith(".docx"):

dept_documents.extend(Docx2txtLoader(file_pa
th).load())

                    else:
                        print(f"  ->
Skipping unsupported file type: {filename}")

                except Exception as e:
                    print(f"  -> Error
reading {filename}: {e}")

        if dept_documents:
            print(f"-> Found
{len(dept_documents)} document(s) in {dept}.
Splitting into chunks...")

            # Split documents into small
readable blocks
            text_splitter =
RecursiveCharacterTextSplitter(chunk_size=10
00, chunk_overlap=200)
            dept_chunks =
text_splitter.split_documents(dept_documents
)

            # Dynamically tag every chunk
with its matching department name
            for chunk in dept_chunks:
                chunk.metadata["department"]
= dept


all_processed_chunks.extend(dept_chunks)
```

```
            print(f"-> Successfully
processed {len(dept_chunks)} chunks for
{dept}.")
        else:
            print(f"-> No readable documents
found in '{dept}' folder.")
    else:
        print(f"\nWarning: Folder not found
for '{dept}' at path: {dept_folder_path}")

# --- 4. Commit Everything to ChromaDB ---
if all_processed_chunks:
    print(f"\nSaving a total of
{len(all_processed_chunks)} chunks to
ChromaDB...")

    vector_db = Chroma.from_documents(
        documents=all_processed_chunks,
        embedding=embeddings,
        persist_directory=PERSIST_DIRECTORY
    )
    print("Database sync complete! The AI's
memory is fully up to date.")
else:
    print("\nNo new documents were found
across any departments. Database remains
unchanged.")
```

- Once you run this script, a new folder called ./local_chroma_db will appear in the project's directory. This holds your embedded vectors. When you are ready to chat with Qwen3.5 again using LangChain, just load the existing ChromaDB instance as a "retriever" and pass the retrieved chucks to the Qwen model as context
- Note: only .pdf, .txt and .docx files will be read because we don't want irrelevant files to be stored in ChromaDB

5. Automate the program so that it runs each day:

    1. Click your Windows start button and type “Task Scheduler”. Hit “Enter”

    2. On the right-hand side panel, click “Create Basic Task”

    3. Name it something clear like “TFF AI Database Sync” and click “Next”

    4. Trigger: select daily and click “Next”. Choose a time you know your device will be on and running

    5. Action: select “Start a program” and click “Next”

    6. Program/script: Type exactly python

    7. Add arguments: type exactly EmbeddedModel.py

    8. Starting in: paste in the path to the folder where your Python script lives:

        C:\Users\User\Downloads\Local LLM-AI Agent\Python Code

    9. Click “Finish”

Note: you may need to make a batch file using the following steps if the above steps are unsuccessful:

1. Create a new text file in the same folder where you have the code

2. Name this file “Run AI Sync.bat” and if Windows warns you about changing the file format, click “Yes” (note: you will need to unhide file extensions before doing this by clicking the “View” tab in File Explorer)

3. Right click the new .bat file and click “Edit” - this will open the batch file in Notepad

4. Write the following code into Notepad:

```
@echo off
cd /d "C:\Users\User\Downloads\Local LLM AI
Agent\Python Code"
python EmbeddedModel.py
pause
```

5. Double-click to run “Run AI Sync.bat” to make sure that it works

Now we need to go and update the Task Scheduler with this updated .txt file:

1. Open Task Scheduler, find your task, right click it and go to “Properties”

2. Go to the actions tab, select your action and hit “Edit”

3. Program/script box: click the “Browse …” button, navigate to the folder where the “Run AI Sync.bat” file is located and select it

4. Add arguments box: delete everything here and leave it blank

5. Click “OK” to save

6. Right click the task and hit “Run”

6. User Interface (LLM Interface)

1. Choose a Python User Interface (UI) Framework for LLMs. I recommend **Streamlit** because it is the industry standard for building AI data and apps quickly. It also has built-in chat elements like st.chat_message and st.chat_input, that make creating an UI similar to ChatGPT’s very easy. As a result, I will show the rest of this section using Streamlit.

2. Install Strealmit and LangChain Chroma the same way you installed ChromaDB:

```
pip install streamlit
pip install langchain-chroma
```

3. Create your UI script, UserInterface.py

```
import json
import streamlit as st
from langchain_openai import ChatOpenAI,
OpenAIEmbeddings
from langchain_chroma import Chroma
from langchain_core.messages import HumanMessage,
AIMessage, SystemMessage

# --- 1. Load Approved Departments from JSON
Config ---
with open("config.json", "r", encoding="utf-8")
as file:
    config = json.load(file)
departments = config["approved_departments"]

# --- 2. Page Configuration ---
# Using your Postimages direct link ensures the
logo works perfectly on any machine
st.set_page_config(page_title="TFF AI Agent",
page_icon="https://i.postimg.cc/fW0RysMZ/TFF-Logo
.jpg")
st.title("Toronto Fresh Food (TFF) AI Agent")
st.subheader("Ask me any questions about your
department at TFF!")

# --- 3. Sidebar: Department Filter ---
st.sidebar.title("Search Settings")
st.sidebar.markdown("Select a department to
filter the documents the AI reads.")
selected_dept = st.sidebar.selectbox("Department
Filter", departments)

# --- 4. Initialize Models & Database ---
@st.cache_resource
def get_llm():
    return ChatOpenAI(
        base_url="http://localhost:1234/v1",
        api_key="lm-studio",
        model="qwen3.5-27b",
```

```
        temperature=0.7
    )

@st.cache_resource
def get_vector_db():
    # Must perfectly match the embedding setup
used in the ingestion script!
    embeddings = OpenAIEmbeddings(

openai_api_base="http://localhost:1234/v1",
        openai_api_key="lm-studio",
        model="nomic-embed-text-v1.5",
        check_embedding_ctx_length=False
    )
    return
Chroma(persist_directory="./local_chroma_db",
embedding_function=embeddings)

llm = get_llm()
vector_db = get_vector_db()

# --- 5. Manage Chat History in Session State ---
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display existing chat history
for message in st.session_state.messages:
    role = "user" if isinstance(message,
HumanMessage) else "assistant"
    with st.chat_message(role):
        st.markdown(message.content)

# --- 6. Handle User Input & RAG Logic ---
if prompt := st.chat_input("Ask a question based
on your documents..."):

    # Add user message to UI and history

st.session_state.messages.append(HumanMessage(con
tent=prompt))
    with st.chat_message("user"):
```

```
        st.markdown(prompt)

    # Generate response
    with st.chat_message("assistant"):
        response_placeholder = st.empty()

        with st.spinner(f"Searching
{selected_dept} documents..."):
            # Retrieve relevant text chunks
matching the prompt AND the active department
metadata
            retriever = vector_db.as_retriever(
                search_kwargs={
                    "k": 3,
                    "filter": {"department":
selected_dept}
                }
            )
            context_docs =
retriever.invoke(prompt)

            # Combine retrieved text blocks into
one single string
            context_text =
"\n\n---\n\n".join([doc.page_content for doc in
context_docs])

            # Create the hidden system
instructions feeding context to the AI
            system_instruction = f"""You are a
helpful AI assistant for the {selected_dept}
department at Toronto Fresh Food (TFF).
            Use the following retrieved context
from the company database to answer the user's
question.
            If the answer is not contained within
the context, simply say that you do not know.

            Retrieved Context:
            {context_text}
            """
```

```
            # Prepend system instruction to the
active ongoing chat conversation
            full_messages =
[SystemMessage(content=system_instruction)] +
st.session_state.messages

        with st.spinner("Generating answer..."):
            # Call LM Studio with the combined
prompt, history, and context
            response = llm.invoke(full_messages)

            # Render response onto the UI

response_placeholder.markdown(response.content)

            # Optional dropdown feature allowing
users to view source documents transparency
            if context_docs:
                with st.expander("View Source
Documents"):
                    for i, doc in
enumerate(context_docs):
                        st.markdown(f"**Source
{i+1}:**")

st.write(doc.page_content)
                        st.caption(f"Metadata:
{doc.metadata}")
                        st.divider()

    # Append the AI's response to the history so
it remembers context next turn

st.session_state.messages.append(AIMessage(conten
t=response.content))
```

4. Run your UI

    - You cannot run this code normally, you must use command prompt. In your command prompt, run:

      ```
      cd C:\Users\User\Downloads\Local LLM AI Agent\Python Code

      python -m streamlit run UserInterface.py
      ```

    - This will automatically open a browser window with your new Chat UI connecting to LM Studio

7. Cloud Solution vs Physical Solution

If we wanted to use the Cloud Solution instead of directly running it from a device, we would need to rent a GPU-enabled virtual machine (VM) and then install the reference engine onto it. You will first have to choose a provider of GPU focused-cloud services or cloud virtual machines. I would recommend **Runpod** because of its second building, wide selection of GPUs (e.g; A100, H100, RTX 4090/5090) and is easy to use for AI workloads. Then you launch a VM (e.g; Ubuntu) equipped with NVIDIA GPUs and finally install LM Studio via the terminal commands on the cloud instance.

If we were to use Runpod to run LM Studio, the process would look something like this:

1. Spin up a base pod

    - Log into Runpod at go to deploy
    - Select a base template with CUDA support (e.g; the official runpod/base template for Ubtunu + CUDA)
    - Select one of the GPUs that has 48 GB of VRAM (we need a buffer if multiple departments are going to use the local LLM) . I recommend **RTX 6000 Ada** because it is considered one of the best single-card solutions for running local LLMs due to its Ada Lovelace architecture that allows for fast interference speeds and high token generation
    - (Important) - click “Customize Deployment” under “HTTP Ports” and enter “1234” (this is the default port LM Studio’s API uses)

- Deploy the pod

2. Connect and install LM Studio CLI

    - Once our pod is running, click “Connect” and open the Web Terminal (or SSH into it if you prefer)
    - Install the LM Studio daemon by running the following command:

      ```
      curl -fsSL https://lmstudio.ai/install.sh | bash
      ```

    - CRITICAL: Symlink the LM studio models folder for your persistent workspace so you don’t lose any downloads on each restart!

      ```
      mkdir -p /workspace/lmstudio_models mkdir -p
      ~/.lmstudio ln -s /workspace/lmstudio_models
      ~/.lmstudio/models
      ```

    - Start the background service daemon:

      ```
      lms daemon up
      ```

3. Download and run the model

    - Find the Hugging Face Repository for Qwen3.5-27B with Q4_K_XL quants (see the link in first section)

    - Download the model using LM Studio CLI:

      ```
      lms get unsloth/Qwen3.5-27B-GGUF:Q4_K_XL
      ```

    - Start the OpenAI-compatiable API server using tmux so it stays alive even if you close the RunPod terminal window:

      ```
      tmux new -d -s lmstudio_server "lms server start"
      ```

4. Connect our API

    - To connect the external apps/scripts we made in the previous steps, we won't use localhost: 1234. Instead, Runplot provides a proxy URL to route external traffic to your Pod’s dashboard

- Click “Connect” and look for the HTTP service link for port 1234
- You will use that URL as your base URL appending /v1 to the end. It will look something like the following:

  `https://<YOUR_POD_ID>-1234.proxy.runpod.net/v1`

In order to share these local LLMs across TFF, we would need to use the following process that differs slightly whether or not we use a physical solution or a cloud solution.

For a physical solution we would use a host device (i.e. a central desktop) which is running LM Studio. Then, we would use the following steps to share the local LLMs across the company.

1. Ensure that LM Studio is downloaded on both the host and client machine

2. Active LM Link

   - Open the sidebar and click on “LM Link” and follow the prompts to log in
   - This creates a secure link across the networks

3. On our host device, load our desired model in the “AI Chat” or in the “Local Server” tab

4. Client access: on your remote device, open the LM link and select “Add a remote machine.” The hosted models will appear as local options, allowing cross-network usage

This is the easiest and most secure way to share the local LLMs across the company if we intended to use one host machine.

For a cloud solution, we would use Runpod and use the following process:

1. Install Tailscale on the Runpod container

   1. Access the Runpod container via JupyterLab Web Terminal

   2. Open a new terminal window

   3. Install Tailscale:

- Run the official one-line installation script:

  ```
  curl -fsSL https://tailscale.com/install.sh | sh
  ```

4. Start the Tailscale daemon (userspace mode):

   - Once installed, start the tailscale dameon. Because Runpod containers might not have `systemed` enabled by default, run this command to start in the background:

     ```
     tailscaled --tun=userspace-networking
     --state=/workspace/tailscaled.state &
     ```

   - Note: saving the file to /workspace/ ensures your pod remembers its Tailscale IP addresses even if you restart the pod!

5. Authenticate and connect

   - Run the following command to connect your Runpod container to your Tailscale network:

     ```
     tailscale up
     ```

   - This will output a URL (e.g; https://tailscale.com)

   - Copy and paste the URL into a browser and log in with someone's Tailscale account to authenticate the container

6. Verify connection

   - To confirm that it is connected and to get the Tailscale IP address of your pod, run the following script:

     ```
     sudo tailscale ip -4
     ```

2. Bind LM Studio to all interfaces

   - When you start your LM Studio server, you must ensure that it listens for external Tailscale traffic. Start it with this flag:

```
lms server start --host 0.0.0.0
```

3. Install and login into **Tailscale** on all client devices

4. Accesses the Runpod LM Studio server using its Tailscale IP address (100.x.x)

    - In the Python codes stated above, you will replace "`http://localhost:1234/v1`" with "`http://100.x.x.x:1234/v1`"

    The reason why it is preferred to use Tailscale over using Runpod's public URL method is that Tailscale is a mesh VPN. Public URLs expose your local service to the entire internet which can increase the risk of data prompt injection attacks, unauthorized access and scanning. A mesh VPN like Tailscale offers a zero-trust approach, restricting access to only authenticated devices. This makes it a more secure format for deploying the local LLMs.

    Generally, running a local LLM on the cloud compared to running a local LLM from a central desktop is more expensive for consistent, high-volume use but is cheaper for occasional, short-term use. This decision comes from whether or not a CAPEX (local hardware purchase) is more preferable than an OPEX (cloud renting) scenario. If you decide to use an OPEX solution, you will have to pay per hour as well, per token or both. While this is cheaper for lower-frequency usage over a short period of time, as usage and time increases renting a GPU becomes more expensive than buying a GPU.

    As for the security of the company, it would be preferable to use the physical solution because none of the company's data will ever leave company/trusted devices. This is preferable to the OPEX solution in which you will have to give TFF's data to a VM provider and have to trust that its security is strong enough to keep your data protected, especially because hackers have an easier time and more incentives to hack into a cloud provider than into your personal computer.

    However, while the cloud solution may be more expensive than the physical solution, it may be more convenient. This is because with a physical solution, you have to ensure that the desktop stays on for all working hours because if it turns off, none of the client machines will be able to use the AI agents. Also, the cloud solution is more flexible because it is not attached to any expensive hardware that needs to be replaced. If we wanted to scale up the business with more powerful AI agents we wouldn't need to buy any new equipment, we would just have to cancel renting the current GPU and start renting a GPU with more VRAM. However, with the cloud solution the IT team would likely have to do more work since they

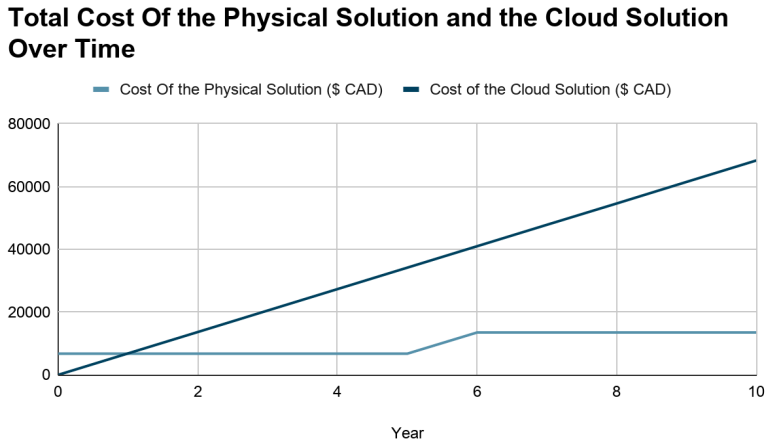
would have to install Tailscale on the Runpod container and ensure that Tailscale is installed on all client machines alongside LM Studio. This is more complex than using the physical solution because LM Link is already built into LM Studio so all the IT team would have to do is make sure that LM Studio is installed on all devices that want to use the AI agents.

What Would Be the Best Physical or Cloud Solution?

If we chose to use the physical solution it would make the most sense to buy a Mac desktop with 64-128 GB of RAM (once again, we need a buffer if all 7 departments are going to be using this Local LLM) and a monitor. The reason we should go with Mac over Windows in this case is because Apple Silicon uses unified memory between the CPU and the GPU, allowing for the GPU to utilize almost this entire pool of memory as VRAM. Also, we are only using these AI agents for casual, chat style interference. This means that we do not need the advantages that NVIDIA GPUs bring at training, high-context processing and diffusion image generation. I personally recommend the **Mac Studio with M3 Ultra Chip, 96 GB of Unified Memory and 1 TB of Storage** PC and **Alienware 34 Gaming - AW3425DWM** monitor. The total cost for this solution is $6,756.26. The best cloud solution would be to use Runpod and rent the RTX 6000 Ada GPU and there would be no starting cost. However, it would cost $0.78 per hour.

Comparing the Best Solutions and Making a Final Decision

The following graph compares the total costs of the solutions over the span of ten years. The graph makes the assumption that the desktop and monitor are deprecating assets that have a useful life of 5 years and then need to be salvaged and replaced but salvage values were not deducted from the total costs. It also makes the assumption that we would be running the AI agents 24/7 on Runpod. Since we are using different RAG pipelines for each department, we do not need to deploy 7 Local LLMs, we only need to deploy one.



As you can see from the graph, the physical solution over the span of ten years is cheaper at $13,512.52 than the cloud solution at $68,328. This means that on average, the cloud solution is about $5,481.59 more expensive per year. As stated above, you can also salvage the PC and make some of your money back but you cannot do this with the cloud solution because you are renting a GPU and never really owned it as an asset. So, after examining the price differentials between both solutions in relation to TFF's revenue and profit margins over the next decade, the cloud solution is not meaningfully more expensive than the physical solution. Therefore, the price should not play a factor in the final decision.

As for what the final decision should be, it comes down to what the company prefers:

- If you prefer better security and easier to set up option at the expense of harder scalability and less flexibility, than the physical solution is best

- If you don't care as much about 100% security and are willing to spend more time setting up Tailscale on each client device for easier scalability and more flexibility, than the cloud solution is best

8. What if We Wanted to Use this Local LLM Model for General Inquiries?

Since Qwen 3.5-27B has already been trained, it will give answers based on its training to general inquiries in a process known as "hallucinating". The reason why it is not a good idea to use an AI model that hallucinates to give answers is

that they are not trained to the most relevant data. For example, the Qwen model we are using is only trained up till 2024, meaning that there will be much more relevant data that exists that the local LLM won't be able to use to give a precise answer. The whole purpose of using this model is to create a secure and stable way for each department at TFF to have an AI support agent that helps to answer their questions based on their department's documents. Trying to make a local LLM that can retrieve data from the internet is out of scope for the RAG pipelines we are using and users would be better off using an generative AI model from the internet such as Gemini Pro if they want to make general inquiries.

However, the AI agents can be good for automating some basic tasks based on their previous training. These takes include: doing mathematical calculations, filling out forms and any other mundane tasks that do not require contemporary data to do. For example, if you want the sum of the costs of a bunch of sushi rolls, you can use the AI agent to calculate the sum for you if the software you are using does not have a calculator. Another relevant example could be that you want to improve the way an email/important document sounds, so you ask the Local LLM to improve your writing. This could be very useful because it allows you to get AI feedback on your writing without having to risk putting important company data on the internet if you were to use a standard LLM like Gemini Pro or ChatGPT.